GUIDE DE RÉVISION IMPRIMABLE — INFORMATIQUE CPGE SUP

Préparation à la 2e année (Spé) • Synthèse Théorie & Pratique Python

Ce guide récapitule l'ensemble des notions fondamentales d'informatique pour réussir la transition entre la 1re année (Sup) et la 2e année (Spé) en CPGE scientifiques (filières MPSI/PCSI/PSI). Il repose strictement sur les méthodes, preuves et complexités de vos cours et TD importés, tout en identifiant clairement à la fin les notions complémentaires à compléter avant les concours.

PARTIE 1 – Bases de Python

1.1 Variables, Types de base & Affectation

1. Définition : En Python, les variables sont des références vers des objets en mémoire. Les types de base incluent int, float, bool et str.

2. Utilisation : **Quand utiliser :** Pour stocker des valeurs intermédiaires et structurer le calcul. | **Quand éviter :** Éviter de réaffecter des types différents à une même variable dans un calcul mathématique.

3. Exemple de code Python commenté :

```
# Affectation et typage dynamique
a = 10         # int
x = 3.14       # float
is_valid = True # bool
```

4. Erreurs classiques / Pièges : Confondre l'affectation simple `=` et le test d'égalité `==`.

5. Astuce à retenir : *Python gère les entiers `int` en précision arbitraire (pas de dépassement de capacité d'entiers).*

1.2 Boucles Iteratives (for et while)

1. Définition : La boucle `for` parcourt une séquence déterminée d'éléments ; la boucle `while` répète un bloc tant qu'une condition booléenne reste vraie.

2. Utilisation : **Quand utiliser :** `for` si le nombre de tours est connu à l'avance ; `while` si l'arrêt dépend d'une condition dynamique. | **Quand éviter :** Éviter `while` sans mettre à jour correctement la variable de contrôle (risque de boucle infinie).

3. Exemple de code Python commenté :

```
# Boucle for avec range
sum_val = 0
for i in range(1, 5):
    sum_val += i

# Boucle while logarithmique
i = 1
while i <= 16:
    i = i * 2
```

4. Erreurs classiques / Pièges : Écrire `while g < d` au lieu de `g <= d` en dichotomie, ou oublier d'incrémenter l'indice.

5. Astuce à retenir : *Pour un `while`, appliquez la méthode des 4 étapes (variable de contrôle, valeur u_k, inéquation d'arrêt, nombre de tours).*

1.3 Fonctions et Portée des Variables (Scope)

1. Définition : Une fonction englobe un bloc d'instructions réutilisable avec des paramètres d'entrée et une valeur de retour (`return`).

2. Utilisation : **Quand utiliser :** Pour modulariser le code et éviter la répétition d'instructions. | **Quand éviter :** Ne pas utiliser de variables globales modifiées à l'intérieur d'une fonction sans paramètre explicitement transmis.

3. Exemple de code Python commenté :

```
def carre(x):
    # Variable locale res
    res = x * x
    return res

val = carre(5)
```

4. Erreurs classiques / Pièges : Oublier l'instruction `return` (la fonction renvoie alors `None` par défaut).

5. Astuce à retenir : *Une fonction s'arrête immédiatement dès le premier `return` rencontré.*

PARTIE 2 – Structures de données (Analyse & Coûts Python)

Tableau comparatif des opérations et complexités sous Python :

Opération Python	Complexité	Commentaire / Mécanisme interne
L[i], len(L)	O(1)	Accès direct en mémoire par pointeur
L.append(x), L.pop()	O(1)	Ajout / retrait amorti en fin de tableau dynamiquement redimensionné
L.insert(0,x), L.pop(0), del L[0]	**O(n)**	**Piège classique :** Décale l'ensemble des n éléments vers la droite ou la gauche
x in L, min(L), max(L), sum(L)	**O(n)**	Parcours séquentiel complet des n éléments de la liste
Slicing L[i:j], L.copy(), list(L)	**O(j - i) / O(n)**	Allocation et **copie physique** d'une nouvelle sous-liste en mémoire
L1 + L2	**O(n1 + n2)**	Création d'une nouvelle liste et copie des deux listes initiales
sorted(L), L.sort()	**O(n log n)**	Tri Timsort hybride optimal

2.1 Slicing, Aliasing et Copie Superficielle vs Profonde

1. Définition : L'aliasing se produit lorsque deux variables pointent vers le même objet mutable. Le slicing `L[:]` effectue une copie superficielle.

2. Utilisation : **Quand utiliser :** Utiliser `L.copy()` ou `L[:]` pour dupliquer une liste sans modifier l'originale. | **Quand éviter :** Ne pas faire `L2 = L1` en pensant créer une nouvelle liste séparée.

3. Exemple de code Python commenté :

```
A = [1, 2, 3]
B = A       # Aliasing : B pointe vers le même objet que A
C = A.copy()# Copie superficielle : nouvelle liste créée
B[0] = 99   # A[0] devient aussi 99 !
```

4. Erreurs classiques / Pièges : Modifier une sous-liste transmise en paramètre en oubliant l'effet de bord sur la liste d'origine.

5. Astuce à retenir : *L'égalité de référence `A is B` teste si deux variables partagent la même adresse mémoire.*

2.2 Piles (LIFO) et Files (FIFO)

1. Définition : Une Pile (Last In, First Out) insère et retire par le haut. Une File (First In, First Out) insère à la queue et retire par la tête.

2. Utilisation : **Quand utiliser :** Pile : parcours en profondeur, gestion des appels récursifs, vérification de parenthésage. File : parcours en largeur. | **Quand éviter :** Éviter d'implémenter une file avec une liste Python standard en faisant `L.pop(0)` qui coûte O(n).

3. Exemple de code Python commenté :

```
from collections import deque
# Pile avec liste
pile = []
pile.append(10) ; v = pile.pop() # O(1)

# File avec deque
file = deque()
file.append(10) ; v = file.popleft() # O(1)
```

4. Erreurs classiques / Pièges : Utiliser `L.pop(0)` dans une boucle de n tours, transformant un algorithme O(n) en $O(n^2)$.

5. Astuce à retenir : *En Python, `collections.deque` offre l'ajout et le retrait en O(1) aux deux extrémités.*

PARTIE 3 – Récursivité (Théorie, Relations & Pile d'Appels)

3.1 Principe, Cas de base et Empilement

1. Définition : Une fonction récursive s'appelle elle-même en résolvant un sous-problème plus petit jusqu'à atteindre un cas de base terminal.

2. Utilisation : **Quand utiliser :** Idéal pour la stratégie 'Diviser pour Régner' et les structures arborescentes. | **Quand éviter :** Éviter les fonctions récursives sans cas de base valide ou provoquant deux appels sur (n-1).

3. Exemple de code Python commenté :

```
def factorielle(n):
    if n == 0:     # Cas de base O(1)
        return 1
    return n * factorielle(n - 1) # Appel récursif
```

4. Erreurs classiques / Pièges : Oublier que chaque appel récursif en attente consomme un cadre sur la pile d'appels (mémoire spatiale O(profondeur)).

5. Astuce à retenir : *La profondeur maximale par défaut sous Python provoque l'erreur `RecursionError: maximum recursion depth exceeded`.*

Tableau des Récurrences Récursives à connaître par cœur en Concours :

Relation de récurrence T(n)	Complexité T(n)	Exemple type en TP / Concours
T(n) = T(n - 1) + O(1)	O(n)	Factorielle récursive, recherche linéaire récursive
T(n) = T(n - 1) + O(n)	O(n²)	Tri par sélection récursif, suppression récursive
T(n) = T(n/2) + O(1)	O(log n)	Recherche dichotomique, exponentiation rapide
T(n) = 2 T(n/2) + O(1)	O(n)	Recherche du maximum par dichotomie
T(n) = 2 T(n/2) + O(n)	O(n log n)	Tri par fusion (Merge Sort)
T(n) = 2 T(n - 1) + O(1)	O(2■)	Tours de Hanoï (à éviter pour grand n !)
T(n) = T(n - 1) + T(n - 2) + O(1)	O(ϕ■) avec ϕ ≈ 1.618	Fibonacci récursif naïf

3.2 Exponentiation Rapide (Récursive et Itérative)

1. Définition : Calcule x■ en O(log n) multiplications au lieu de O(n) en élevant au carré selon la parité de n.

2. Utilisation : **Quand utiliser :** Indispensable en arithmétique pour élever des entiers ou des matrices à des puissances élevées. | **Quand éviter :** Ne pas utiliser la boucle naïve `for i in range(n): r *= x` qui prend O(n) temps.

3. Exemple de code Python commenté :

```
# Version récursive O(log n) temps, O(log n) espace
def puissance_rec(x, n):
    if n == 0: return 1
    if n % 2 == 0:
        y = puissance_rec(x, n // 2)
        return y * y
    else:
        y = puissance_rec(x, (n - 1) // 2)
        return y * y * x

# Version itérative O(log n) temps, O(1) espace
def puissance_iter(x, n):
    r = 1
    while n > 0:
        if n % 2 == 1: r = r * x
        x = x * x ; n = n // 2
    return r
```

4. Erreurs classiques / Pièges : Oublier de mémoriser `y = puissance(x, n//2)` et écrire `puissance(x, n//2) * puissance(x, n//2)`, réinjectant deux appels et retombant en O(n) !

5. Astuce à retenir : *La version itérative économise la pile d'appels et fonctionne en espace constant O(1).*

PARTIE 4 – Algorithmique (Recherche, Tris & Preuves)

4.1 Recherche Dichotomique dans une Liste Triée

1. Définition : Localise un élément dans une liste préalablement triée en divisant par deux l'intervalle de recherche à chaque étape.

2. Utilisation : **Quand utiliser :** À utiliser uniquement lorsque la liste d'entrée est strictement TRIÉE. | **Quand éviter :** Ne JAMAIS appliquer la dichotomie sur une liste non triée (résultat complètement faux).

3. Exemple de code Python commenté :

```
def dichotomie(x, L):
    g, d = 0, len(L) - 1
    while g <= d:   # Attention au <= !
        m = (g + d) // 2
        if L[m] == x: return m
        elif L[m] < x: g = m + 1
        else: d = m - 1
    return None
```

4. Erreurs classiques / Pièges : Écrire `while g < d` au lieu de `g <= d` (oublie le dernier élément) ou `d = m` au lieu de `d = m - 1` (boucle infinie possible).

5. Astuce à retenir : *Complexité temporelle au pire : O(log n). Complexité au meilleur : O(1) si x est exactement au milieu m.*

4.2 Tri par Sélection et Tri par Insertion

1. Définition : Tris itératifs quadratiques O(n²). La sélection cherche le plus petit élément restant ; l'insertion insère chaque élément à sa place dans la partie triée.

2. Utilisation : **Quand utiliser :** Utiles pour de très petites listes ou des listes presque triées (pour le tri par insertion : meilleur cas en O(n)). | **Quand éviter :** Éviter sur de grands ensembles de données (n > 10 000 prend plusieurs secondes sous Python).

3. Exemple de code Python commenté :

```
def tri_insertion(L):
    n = len(L)
    for i in range(1, n):
        x = L[i] ; j = i - 1
        while j >= 0 and L[j] > x:
            L[j + 1] = L[j] ; j -= 1
        L[j + 1] = x
```

4. Erreurs classiques / Pièges : Pour le tri par insertion : la complexité au pire est O(n²) (liste inversée) mais au meilleur cas O(n) (liste déjà triée).

5. Astuce à retenir : *En Python (~10■ opérations/sec), n = 10■ est la limite pratique des tris en O(n²).*

4.3 Tri par Fusion (Diviser pour Régner - O(n log n))

1. Définition : Découpe la liste en deux moitiés, les trie récursivement puis les fusionne en une seule liste triée en O(n).

2. Utilisation : **Quand utiliser :** Le tri optimal pour garantir une complexité en O(n log n) dans TOUS les cas (meilleur, moyen, pire). | **Quand éviter :** Attention au coût mémoire : l'implémentation usuelle n'est pas 'en place' et requiert O(n) mémoire auxiliaire.

3. Exemple de code Python commenté :

```
def fusion(A, B):
    res = [] ; i = j = 0
    while i < len(A) and j < len(B):
        if A[i] <= B[j]: res.append(A[i]) ; i += 1
        else: res.append(B[j]) ; j += 1
    res.extend(A[i:]) ; res.extend(B[j:])
    return res

def tri_fusion(L):
    if len(L) <= 1: return L
    m = len(L) // 2
    return fusion(tri_fusion(L[:m]), tri_fusion(L[m:]))
```

4. Erreurs classiques / Pièges : Oublier le coût du slicing `L[:m]` et `L[m:]` qui effectue des copies en O(n).

5. Astuce à retenir : *Grâce à la condition `<=` lors de la fusion, le tri par fusion est STABLE (conserve l'ordre des éléments égaux).*

PARTIE 5 – Outils Scientifiques & Notions Manquantes

Bilan de Couverture de vos Sources (Analyse d'Ingestion)

Vos documents importés (ch01-Complexité-complet, fiche_complexite, Fiche_methode_DS1_PSI, TD1) couvrent de manière ULTRA-DÉTAILLÉE et EXHAUSTIVE l'Algorithmique, la Complexité Temporelle/Spatiale et les Tris. Cependant, certaines parties théoriques globales du programme complet de Sup ne figurent pas dans vos fichiers.

Liste des Notions du Programme de Sup absentes de vos sources (À compléter pendant l'année de Spé) :
• **Outils Scientifiques Python :** Manipulation de tableaux multidimensionnels avec **NumPy** (arrays, vectorisation, slicing 2D) et tracés de courbes avec **Matplotlib** (`plt.plot`, `plt.show`, légendes).
• **Bases de Données & SQL :** Modèle relationnel, requêtes d'extraction `SELECT ... FROM ... WHERE`, jointures `JOIN` et agrégations `GROUP BY`.
• **Structures Avancées :** Représentation formelle des Arbres Binaires et Graphes (matrice d'adjacence, listes de successeurs).
• **Gestion des Fichiers :** Manipulation des fichiers texte avec `open('fichier.txt', 'r')` et `read()/write()`.

PARTIE 6 – Synthèse Finale : Réflexes & Glossaire Concours

Tableau 'Si le sujet demande X → J'utilise Y' (Réflexes Concours) :

Si l'énoncé de concours demande...	Réflexe d'algorithme / Structure à adopter
Calculer la puissance n-ième x■ ou élever une matrice	**Exponentiation rapide** (O(log n) temps)
Chercher un élément dans une liste *déjà triée*	**Recherche dichotomique** (O(log n) temps)
Trier une liste avec la meilleure garantie de complexité	**Tri par fusion** (O(n log n) garanti au pire cas)
Évaluer un polynôme P(x) = a■ + a■x + ... + a■x■	**Méthode de Horner** (O(n) multiplications au lieu de O(n²))
Gérer un historique d'actions ou un parenthésage	**Structure de Pile (LIFO)** avec `append` et `pop`
Justifier la complexité d'une fonction récursive	Écrire d'abord **T(n) = a T(sous-problème) + f(n)** puis résoudre

Glossaire des Termes Techniques (Français / Anglais) :

Terme Français	Equivalent Anglais	Définition synthétique

Complexité temporelle	Time complexity	Nombre d'opérations élémentaires en fonction de n
Complexité spatiale	Space complexity	Mémoire auxiliaire utilisée (pile d'appels incluse)
Grand O (Asymptotique)	Big O notation	Majoration du coût pour de grandes valeurs de n
Pile	Stack (LIFO)	Dernier entré, premier sorti
File	Queue (FIFO)	Premier entré, premier sorti
Tri par fusion	Merge sort	Tri récursif par division et recombinaison O(n log n)
Dichotomie	Binary search / Divide and conquer	Division successive par 2 de la zone d'étude